Quarry Lux Control-Panel Applet: Product Requirements

Punt Labs

August 2026

Contents

Chapter 1

Product Requirements

1.1 Overview

Every Punt Labs tool's configuration is repo-scoped by convention (punt-kit's tool-enable-disable standard): quarry's own capture behavior for a given repository lives in that repository's `.punt-labs/quarry/config.md`, written once by `quarry enable` and, today, only ever edited by hand or by re-running `enable`. This document specifies a lux applet that makes those settings live-editable from the lux menu bar, for the repository the user's current Claude Code session is actually working in.

This is a genuinely different kind of applet from the search applet (`quarry-lux-search-applet-prd.tex`): it is session-bound, spawned by the session-start hook and tied to the session's own working directory via a parent-pid watch, the same species as `lux-beads` and `vox-panel` (DES-063, lux repo). A machine-global, daemon-embedded applet has no single "current repository" to resolve these settings against; a session does, because it has the session's own shell and working directory, exactly why `lux-beads` reads `bd` from the session's own repo rather than a fixed location.

1.1.1 Priorities and phasing

[P1] is read-and-toggle for the three settings `config.md` already supports today (`session_sync`, `web_fetch`, `compaction`), for the repository the current session is in. [P2] extends to any additional repo-scoped setting that proves worth surfacing here once [P1] ships (see § 1.1.3), and to any cross-session consistency concerns that only show up with more than one session open on the same repository at once.

1.1.2 Goals

- Let a user see and change this repository's quarry capture settings without opening `config.md` in an editor or remembering the YAML frontmatter's exact keys.
- Never let a user believe a setting changed when it did not — what they see on screen after a click is always what is actually true on disk, not an optimistic guess ahead of it. (The atomicity and concurrency-safety this requires of the implementation, modeled on `VoxPanelService`'s persist-then-reflect commit pattern, is an engineering guarantee in service of this goal, not the goal itself; see **FR-7**–**FR-9**.)
- Surface a write failure as a specific, actionable notice rather than a silently reverted toggle.

1.1.3 Non-Goals

- Enabling or disabling quarry for the repository at all. That is `quarry enable/disable`'s job — a CLI-driven, git-tracked, per-repo policy decision (tool-enable-disable.md §2.7), deliberately not a lux toggle. This applet only appears, and only has anything to control, once a repository is already enabled.
- Search, or any view into indexed content. That is the companion search applet's job (`quarry-lux-search-applet-prd.tex`).
- Daemon-level status or control (health, restart, database switching). That stays `quarry-menubar`'s job, and is explicitly out of the companion search applet's scope too — this applet does not duplicate it either.
- The shadow-repo setting `config.md` already supports (pushing redacted captures to a private shadow repo). It is off by default, more involved to toggle safely (it names a remote, acknowledges an unverified-private state), and is deferred to [P2] pending its own use case, not silently folded into [P1]'s three simple booleans.

1.1.4 Personas

The working session owner Has a Claude Code session open in a quarry-enabled repository and wants to quickly quiet or restore one capture behavior — most plausibly turning off web-fetch capture before researching something sensitive, or off compaction capture before a session they do not want transcribed. Primary persona for [P1].

The multi-repo user Has several quarry-enabled repositories and different capture preferences per repository (e.g. compaction capture on for a personal project, off for a client repository), and wants each session's panel to reflect *that* repository's settings, never another session's.

1.2 Use Cases

Each use case names the actor, the trigger, the expected flow, and what "done well" looks like. Functional requirements in §1.3 cite the use cases they satisfy.

UC-1 Open the panel for the current session's repository. [p1] *Actor: the working session owner.* A Claude Code session is open in a quarry-enabled repository; the user clicks the applet's menu entry. *Done well:* the panel shows *this* repository's current settings, read fresh enough that a change made by hand (editing `config.md` directly, or via `quarry enable` re-run) since the panel last loaded is reflected, not stale.

UC-2 Toggle a capture setting off. [p1] *Actor: the working session owner.* The user clicks a setting's toggle (e.g. `web_fetch`) to turn it off. *Done well:* the change is written to `config.md` and confirmed back to the panel before the toggle visually settles into its new state — the panel never shows an optimistic state the write has not actually confirmed, matching `VoxPanelService`'s persist-then-reflect pattern.

UC-3 Toggle a capture setting back on. [p1] *Actor: the working session owner.* Symmetric to **UC-2**. *Done well:* same guarantee in both directions; no setting is easier to turn off than to restore.

UC-4 A write fails. [p1] *Actor: the working session owner.* The toggle click happens but the write to `config.md` fails (disk full, permissions, the file was deleted out from under the session). *Done well:* the panel shows the setting in its *actual* (unchanged) state and a specific notice naming what failed, matching `PanelNotice`'s write-failure pattern — never

a toggle that silently reverts with no explanation, and never a toggle that stays visually "on" when the write that would have made it true did not happen.

UC-5 Open the panel with no quarry session context. [p1] *Actor: any user.* The applet is run unattended (no `--session-pid`, matching `lux-beads`' and `vox-panel`'s own `unattended()` path for a developer running it by hand) or the working directory the session started in is not a quarry-enabled repository. *Done well:* the panel says plainly that there is nothing to control here (not enabled, or no repository context) rather than showing three toggles that silently do nothing when clicked.

UC-6 Two sessions, two repositories, two panels. [p1] *Actor: the multi-repo user.* The user has Claude Code sessions open in two different quarry-enabled repositories at once, each with its own panel instance. *Done well:* each panel shows and controls only its own session's repository; a change in one never appears in, or affects, the other's panel.

UC-7 The session ends. [p1] *Actor: any user.* The Claude Code session that spawned the panel closes (cleanly, or the process dies). *Done well:* the panel's process ends with it (the parent-pid watch, matching `lux-beads/vox-panel` exactly), and the Hub's menu entry for it disappears rather than lingering as a dead click target.

UC-8 Edit config.md by hand while the panel is open. [p2] *Actor: the working session owner.* The user (or another process) edits `config.md` directly while the panel is showing stale values. *Done well:* the panel's next refresh (on its next click, at minimum) picks up the external edit rather than overwriting it with its own stale in-memory state on the next toggle.

1.3 Functional Requirements

Requirements are grouped by capability area. Each cites the use case(s) it exists to satisfy and carries the same [P1]/[P2] phase tag.

1.3.1 Applet lifecycle

FR-1 [p1] The applet shall be spawned by the Claude Code session-start hook, bound to that session's process id, and shall terminate when that session's process ends (parent-pid watch), matching `lux-beads/vox-panel`'s `SessionClaim`/`SessionWatch` composition exactly. (**UC-7**)

FR-2 [p1] The applet shall support an unattended (`--session-pid`-less) invocation for manual/developer use, claiming nothing and outliving nothing, matching `lux-beads/vox-panel`'s `unattended()` path. (**UC-5**)

FR-3 [p1] Each running instance of the applet shall resolve and control only its own spawning session's repository; no shared state shall leak between two concurrently running instances for different repositories. (**UC-6**)

1.3.2 Reading settings

FR-4 [p1] On open, the applet shall read the current repository's `.punt-labs/quarry/config.md` and display the live state of `session_sync`, `web_fetch`, and `compaction`. (**UC-1**)

FR-5 [p1] If the current repository has no `.punt-labs/quarry/config.md` (quarry not enabled there), the applet shall say so explicitly and show no toggles, rather than showing toggles that write to a file that does not exist. (**UC-5**)

FR-6 [**p2**] The applet shall re-read the config file on demand (at minimum, on the next user-initiated refresh) so an external edit is not silently overwritten by the panel's own next write. (**UC-8**)

1.3.3 Changing settings

FR-7 [**p1**] Clicking a setting's toggle shall persist the change to `config.md` and only then update the displayed state to match what was actually persisted — never an optimistic update ahead of the confirmed write. (**UC-2**, **UC-3**)

FR-8 [**p1**] A failed write shall leave the displayed setting at its last-confirmed (unchanged) value and shall show a specific notice naming what failed, distinct from a generic error. (**UC-4**)

FR-9 [**p1**] Every write shall be atomic with respect to the in-memory state it updates — two clicks in quick succession (or a click racing an external edit) shall never leave the panel showing a state that was never actually written, matching `VoxPanelService`'s lock-serialized commit pattern. (**UC-2**, **UC-4**, **UC-8**)

1.4 Non-Functional Requirements

NFR-1 [**p1**] The applet's own process footprint shall be negligible enough that running one per open Claude Code session (the multi-repo persona routinely has several) is not a practical resource concern, matching `lux-beads`' and `vox-panel`'s existing session-bound footprint.

NFR-2 [**p1**] A config write shall complete (success or a reported failure) within a duration a user experiences as immediate — a toggle click should never leave the UI in an ambiguous "pending" state for a perceptible interval under normal conditions.

1.5 Success Metric

Unlike the companion search-applet document, this feature extends an already-used mechanism (`config.md`, the session-start hook) rather than adding a wholly new surface, so the evidence bar is lower — but a falsifiable claim still matters. The metric: once shipped, a non-trivial share of `config.md` auto-capture edits happen through this panel rather than by hand-editing the file. If toggles never happen through the panel while hand-edits continue, the panel has not actually replaced the friction it was built to remove, and [P2] (additional settings beyond the three existing flags) is not justified until that is understood.

1.6 Open Questions

1. Exact menu entry shape and label — a single "Quarry" entry per session (mirroring `vox-panel`'s single "Vox" entry), or something that also names the repository, given a user may have several open at once (**UC-6**)?
2. **Elevated by peer review; carried forward as Chapter 2, §2.5 Decision 3.** Is there a case for surfacing *which* setting most recently changed (an audit trail, even a short one), or is the current-state view (**FR-4**) sufficient for [P1]? This question was originally scoped as a nice-to-have; Chapter 2 found it is not one — §1.6's success metric cannot be measured without some form of it.

3. **Still open; carried forward as Chapter 2, §2.5 Decision 4.** Should the shadow-repo setting (deferred, § 1.1.3) be scoped as a [P2] addition to *this* applet, or as its own separate control surface given its extra safety considerations (naming a remote, an unverified-private acknowledgment)?

4. **Resolved.** Packaging follows `vox-panel`'s own precedent directly: `vox/pyproject.toml:55` registers `vox-panel = "punt_vox.panel.applet:app"` as a console-script entry point inside vox's own existing package — not a separate repository, not a separate distribution. Quarry's control-panel applet ships the same way: a new console script (`quarry-panel`, naming TBD) inside quarry's existing package, pointing at its own `applet:app`. See Chapter 2 for the full packaging decision.

Chapter 2

Technical Design Solution

This chapter resolves the open questions raised in Chapter 1 against the actual code of `lux-beads` and `vox-panel`, the two existing session-bound applets this one is modeled on directly. Nothing here is speculative where the cited source settles the question.

2.1 Component overview

A new `QuarryPanelApplet` (naming TBD), packaged as a console script inside quarry's existing distribution (`quarry-panel = "quarry.panel.applet:app"`, following `vox-panel`'s exact precedent, `vox/pyproject.toml:55`), spawned by the Claude Code session-start hook with `--session-pid`. Structurally identical to `VoxPanelApplet`/`BeadsApplet` (`vox/src/punt_vox/panel/applet.p` `lux/src/punt_lux/applets/beads.py`): a `typer` app, a `main()` command taking `--session-pid` (0 meaning unattended), and a class composing an `AppletProgram` from a `SessionClaim`/`NoClaim`, a leg, and a `SessionWatch`/`NoSession` via `for_session()`/`unattended()` classmethods. Model this file directly on `vox-panel`'s `applet.py`, including its identity-naming caveat (`vox/src/punt_vox/panel/ap` the program token must ride in front of the session pid when deriving the applet's identity, or a second session-bound applet in the same repository (this one, alongside a future or coexisting `lux-beads` instance) would seed an identical connection hash and the second to connect would silently steal the first's Hub connection.

2.2 Settings service: the VoxPanelService precedent

`VoxPanelService` (`vox/src/punt_vox/panel/service.py`) is the direct model for this applet's own settings service, not a loose inspiration — read it in full before implementing. Three mechanisms to reuse exactly:

Persist-then-reflect, under one lock `_commit(field, value, update)` (`service.py:176-183`) writes to the config store first, then updates the held in-memory state, both inside one `threading.Lock` — satisfying **FR-7** and **FR-9** directly. A quarry equivalent commits one of the three `auto_capture` flags to `config.md` the same way.

Write-failure notice, not silent revert `PanelNotice` (referenced throughout `service.py`, e.g. `recover_from_write_failure` at line 125) carries a specific, composed reason for a failed write, read back into the pushed scene rather than silently reverting the toggle. Satisfies **FR-8** directly.

Re-check under the lock after any out-of-lock I/O `_commit_provider`'s pattern (`service.py:243-304`) of fetching outside the lock, then re-checking state under the lock before committing, applies even though quarry's config write is local-disk I/O rather than a daemon round

trip: a second click racing the first's write must not clobber it with stale pre-fetch state. Satisfies **FR-9**'s atomicity requirement precisely, not just in spirit.

2.3 What the quarry version does *not* need

`VoxPanelService` carries real complexity (`_commit_provider`/`_commit_model`'s cascade logic, roster fetches over the network, a six-topic `PanelTopic` enum — `NOTIFY`, `MIC_MODE`, `VOICE`, `VOICE_PREVIEW`, `PROVIDER`, `MODEL`, `vox/src/punt_vox/panel/topics.py:28-33`) that this applet's three independent boolean flags do not require — `session_sync`, `web_fetch`, and `compaction` (per `src/quarry/enable.py`'s `_CONFIG_TEMPLATE`) have no cascading defaults between them and no roster to fetch. The quarry service is simpler: one topic per flag, or one topic carrying a field name, is sufficient — do not import `VoxPanelService`'s cascade machinery wholesale where the simpler three-independent-flags shape does not need it.

2.4 Reading and writing config.md

Reuse `src/quarry/enable.py`'s `_CONFIG_TEMPLATE` field names and YAML frontmatter shape directly — this applet reads and writes the same file `quarry enable` already creates, never a second config surface. **Correction, peer review:** the existing frontmatter *reader* is not in `enable.py` (which only holds the static template string and an exclusive-create writer, `_write_project_config`, that refuses to touch an existing file) — it is `load_hook_config()`/`HookConfig` in `src/quarry/_stdlib.py`, built on the hand-rolled, stdlib-only parser `Frontmatter` in `src/quarry/_frontmat` Reuse *that* reader, not anything in `enable.py`.

No write-back mechanism exists anywhere in quarry today, and this is a real gap, not a citation nit. `Frontmatter` and `HookConfig` are read-only — neither has a `dump`, `write`, or `serialize` method (confirmed: no such method exists in either module). **FR-7**–**FR-9** require a genuine read-modify-write that changes exactly one field and preserves everything else in the file — comments, and in particular a user's hand-edited `shadow:` block, which § 1.1.3 explicitly keeps out of this applet's own writes but which a naive whole-file-regenerate-from-`_CONFIG_TEMPLATE` write would silently destroy. This is a real design gap with real user-facing risk and is named explicitly in "Decisions required," below — it is new code this applet needs, not reuse of something that already exists.

2.5 Decisions required before implementation

1. **Session-repo resolution.** `vox-panel` resolves its repo-scoped config directory via `find_config_dir()` (`vox/src/punt_vox/panel/applet.py:93`). Quarry's equivalent — resolving *which* repository's `.punt-labs/quarry/config.md` to read — needs its own lookup, presumably from the session's working directory the session-start hook already knows. Confirm quarry has (or needs) an equivalent to `find_config_dir()` before implementation starts.
2. **Menu entry naming.** Chapter 1's open question (a single "Quarry" entry per session, vs. one naming the repository) is not resolved here — it is a product decision (**UC-6**), not a technical one, and stays open pending the operator's call.
3. **Write provenance, required by the Success Metric.** Peer review found that §1.6's success metric — a non-trivial share of `config.md` edits happening through this panel rather than by hand — cannot be measured by the design in §2.2 as written: a `VoxPanelService`-style persist-then-reflect write does not, by itself, distinguish a panel-driven write from a hand edit that happened to touch the same field. This needs the (previously optional-seeming) audit-trail question above resolved as a real decision, not deferred: at minimum,

the write path needs to record *something* attributable to this applet — a comment line in the YAML frontmatter, a sibling marker file, or an event logged elsewhere — before the metric is observable at all. Design this alongside whatever the audit-trail decision above settles on, not separately.

4. **Shadow-repo setting scope.** Chapter 1's open question (a [P2] addition to this applet, or its own surface) is not resolved here; it needs its own use case and functional requirements if it is folded into this applet's scope, given its extra safety considerations (§ 1.1.3).

5. **Config.md write-back mechanism, required before FR-7–FR-9 can be implemented at all.** No component in quarry today can rewrite one field of `config.md` while preserving the rest of the file. `Frontmatter`/`HookConfig` (`src/quarry/_frontmatter.py`, `_stdlib.py`) are read-only; `enable.py`'s writer only creates a fresh file and refuses to touch an existing one. This applet needs a genuine round-trip-preserving write path built new — specifically one that does not silently discard a user's hand-edited `shadow:` block or comments when toggling one of the three `auto_capture` flags. Whether this write path lives in `_frontmatter.py` (extending the existing parser with a matching serializer) or is new, applet-local code is an open implementation decision; that it must exist and must be round-trip-preserving is not optional.

2.6 Known unknowns

Whether **FR-6**'s re-read-on-refresh ([P2]) can reuse `VoxPanelService.refresh()`'s resync pattern unchanged, or needs a different trigger given quarry's config file has no daemon to poll against (vox's resync re-reads *and* calls `voxd`; quarry's would only ever re-read a local file), is untested and should be confirmed once [P1] implementation starts, not assumed identical.